Hackathon Deck Evaluator - Reference System Design

Hackathon Deck Evaluator – Reference System Design (Rs. 0 budget)

Design goal: a system that can go from v0 (SIH-only MVP) to v2 (multi-event, SPOC dashboards) without ever leaving free tiers, and that only starts costing money at a scale where you'd have leverage to charge for it (freemium conversions, college partnerships).

Guiding rule: prefer **provider "always free" tiers** (not trial credits that expire) – GCP Cloud Run/Functions, Firebase Spark plan, Supabase free project, Upstash free plan, Groq free API, Vercel Hobby. Trial credits (GCP's Rs. /$300, AWS free trial) are a buffer, not the plan.

---

1. High-level architecture

Five tiers, matching your original doc's layering, mapped onto specific free services:

1. **Client** – student's browser
2. **Frontend** – Next.js on Vercel (Hobby tier, free forever for non-commercial-scale traffic)
3. **App/data layer** – Firebase (Auth + Firestore + Storage) – you already know this stack
4. **Processing engine** – a single Python container on Cloud Run (parsing, scoring, embeddings, LLM orchestration)
5. **Intelligence layer** – free-tier LLM APIs + a vector store + a cache

```
Browser -> Next.js (Vercel) -> Firebase (Auth/Firestore/Storage)
                            -> Cloud Run (Python: parser + scorer)
                                 -> LLM API (Groq / Gemini free
tier)
                                 -> Vector store (Supabase pgvector)
                                 -> Cache (Upstash Redis free tier)
```

The interactive diagram above shows this same graph – this doc is the "why" behind each box.

---

2. Component-by-component reasoning

Frontend – Next.js on Vercel

You already build in Next.js/TS, so this is zero new surface area. Vercel's free Hobby tier gives you 100GB bandwidth/month and serverless function execution – enough for the v0 target of 50-100 teams and well into v1. Deploy the drag-and-drop upload UI, results panel, and (v1) the Q&A chat interface here. Keep heavy work

(parsing, scoring, embeddings) off Vercel functions – their execution time limits (10s on Hobby) will choke a PPTX parse + multi-shot LLM call.

Auth + structured data – Firebase (Spark plan)

Since you already know Firebase: use **Firebase Auth** (free, no cap that matters at this scale) for team/student login, and **Firestore** for team records, scores, and submission metadata. Spark plan gives 1GiB storage and 50K reads / 20K writes / 20K deletes per day – at 50-100 active teams doing a handful of uploads each, you're nowhere close.

File storage – Firebase Storage

5GB free storage, 1GB/day download. PPTX/PDF files are typically 5-20MB, so 5GB holds hundreds of decks. Auto-delete after 7 days (matches your own spec) using a **scheduled Cloud Function** (Firebase's free scheduler) so storage never creeps toward the cap – this is what keeps this component free indefinitely rather than just "free until decks accumulate."

Processing engine – Cloud Run (Python, Dockerized)

This is the one component worth being deliberate about, since it's where `python-pptx`, `pdfplumber`, embedding models, and LLM orchestration all live.

- **Why Cloud Run over Firebase Cloud Functions for this part:** Cloud Run's always-free tier (2 million requests/month, 360,000 GB-seconds, 180,000 vCPU-seconds) is more generous for CPU-heavy work like parsing and local embeddings, and – since you already know Docker – you get full control over the Python environment (no cold-start dependency surprises with binary packages like `python-pptx`).
- Runs as one container exposing a few endpoints: `/parse`, `/score`, `/embed`, `/qa`. At v0 this can be a single monolithic service; split into separate Cloud Run services only if one endpoint's traffic pattern starts dominating (e.g., Q&A chat in v1 has different latency needs than scoring).
- Scale-to-zero is the free-tier trick here: Cloud Run bills (and counts against the free quota) only while a request is being processed, so an idle tool between hackathon seasons costs literally nothing.

LLM calls – Groq and/or Gemini free tiers

Two free options, use them for different jobs: - **Groq** (Llama 3.x models) – free, extremely fast inference, good for the rubric multi-shot scoring pass (Section 3 math below) and the judge Q&A simulation where latency matters for a "conversational" feel. - **Gemini 2.0/2.5 Flash free tier** – ~1,500 requests/day free, better instruction-following for the more nuanced content analysis (jargon density, narrative flow) in v1. - Route both through the Cloud Run service so you can swap/fallback between them without touching the frontend – treat "which LLM" as an internal implementation detail, not an architectural commitment.

Vector store – Supabase pgvector (free project)

For the v1 originality check (embedding past-winning SIH decks and comparing cosine similarity): Supabase's free tier gives 500MB Postgres storage with pgvector built in – enough for tens of thousands of sentence-embedding vectors from scraped SIH repos. This also

gives you a free relational Postgres if you ever want to move off Firestore for the more relational v2 data (teams, colleges, SPOC accounts) without paying for a second service. - Embeddings themselves: run `sentence-transformers` (e.g. `all-MiniLM-L6-v2`) **inside the same Cloud Run container** – CPU inference is fast enough for this model size and costs nothing beyond the compute you're already paying zero for. Avoids a paid embeddings API entirely.

Cache – Upstash Redis (free tier)

10,000 commands/day, 256MB – plenty for the "same team re-uploads same deck twice, don't re-score" cache from your original spec, and for rate-limiting free-tier abuse.

Past-deck corpus (v1 originality engine)

Build via a **scheduled GitHub Action** (free, unlimited minutes on public repos / 2,000 min/month private) that hits the GitHub REST API (free, 5,000 req/hr authenticated) searching `sih`/`smartindiahackathon` tags, and writes results + embeddings into Supabase. This is a nightly/weekly cron, not a live dependency – keeps Cloud Run lean.

CI/CD

GitHub Actions free tier for both the Next.js app (deploy to Vercel via their GitHub integration, which is itself free) and the Cloud Run container (`docker build` -> push to Artifact Registry free tier -> deploy).

Domain

Skip a paid domain at v0/v1 – use the free `*.vercel.app` subdomain. When you're ready for a custom domain, the **GitHub Student Developer Pack** (you're at RVCE, you qualify) includes a free `.me` domain via Namecheap and other student perks worth checking before paying for anything.

---

3. Scoring pipeline (matches your v0 math, mapped to infra)

```
Score = Sum(w_i x s_i)  for i in [Innovation, Feasibility,
Relevance, Scalability, Impact, Clarity]
Final = (Score / 6) x 100
```

Execution flow on Cloud Run: 1. `/parse` – `python-pptx`/`pdfplumber` extracts slide text + order -> structured JSON, cached in Firestore. 2. `/structure-check` – rule-based check against the hardcoded 6-slide SIH template (no LLM call needed – cheapest possible check, do this first so garbage decks fail fast before burning an LLM call). 3. `/score` – one Groq multi-shot call per axis (or one call scoring all 6 axes in a single structured-JSON prompt, which is cheaper and stays under free-tier daily limits far longer than 6 separate calls). 4. Result written to Firestore, cached in Redis keyed by a hash of the uploaded file so a re-upload short-circuits steps 1-3.

---

4. Free-tier ceilings – what breaks first, and when

Component	Free limit	v0 target (50-100 teams)	Breaks around
Cloud Run	2M requests/mo	~few hundred requests	Thousands of teams/month
Firestore	50K reads, 20K writes/day	Low hundreds	Sustained multi-college v2 traffic
Firebase Storage	5GB, 1GB/day download	Comfortable with 7-day auto-delete	Long-term storage without deletion job
Gemini free tier	~1,500 req/day	Fine if Groq handles bulk scoring	High-volume v2 with many colleges live simultaneously
Groq free tier	Rate-limited, no hard $ cost	Fine	Rate limits, not dollars – just slows down, doesn't charge you
Supabase	500MB DB	Tens of thousands of embeddings	Multi-year corpus growth
Upstash Redis	10K commands/day, 256MB	Comfortable	High-traffic launch week
Vercel Hobby	100GB bandwidth/mo	Comfortable	Viral spike during a launch

The honest reading: at your v0/v1 target numbers (50-100 teams, then a few hundred through Unstop/IIT distribution), **you will not hit a single paywall**. The first thing likely to force a decision is Vercel Hobby's commercial-use restriction if you start actively monetizing (their ToS, not a technical limit) – at that point a Rs. 0 Cloudflare Pages migration is a valid free alternative, or you've validated enough to justify Vercel Pro.

---

5. Roadmap alignment

- **v0 (SIH-only MVP):** Next.js + Vercel, Firebase Auth/Firestore/Storage, one Cloud Run container (parse + rule-based structure check + Groq scoring). No vector DB yet – not needed until originality checking.
- **v1 (Q&A + originality):** add Supabase pgvector + local sentence-transformers embeddings in the same Cloud Run container, add the GitHub Action scraper, add Upstash Redis caching, add the Q&A endpoint (Groq, low-latency).
- **v2 (multi-event, SPOC dashboards):** add a rubric-ingestion endpoint (LLM extracts criteria from a pasted URL/rubric text), add a SPOC-facing dashboard route in the same Next.js app gated by Firebase custom claims (college-admin role), start considering Supabase Postgres as the analytics store for the SPOC trend views since it's relational and already in your stack by then.

Nothing in this path requires provisioning a new paid service – v2's only "new" infra is a dashboard route and a rubric-ingestion prompt, both of which run on components already in place.